
Learning Letters and Words in ASL Using Keypoint Sequences

Madelyn Redick
Northeastern University
Boston, MA

Nicole Ni
Northeastern University
Boston, MA

Dhruv Jatkar
Northeastern University
Boston, MA

1 Introduction

The motivation behind this project is to build automated tools that can help bridge accessibility gaps for deaf and hard of hearing communities to recognize American Sign Language (ASL) letters and words from images and videos. Accurate ASL recognition has applications in real-time translation, human computer interaction, and educational technology, but remains a challenge because ASL is a rich visual language rather than a spoken or textual one.

The primary challenge in American Sign Language classification is the capture and interpretation of spatial and temporal linguistic information. ASL uses a variety of articulatory information such as hand shape, location, orientation, and movement. Because these features must be understood both within a single frame and across time, static image models alone are insufficient and sequence models alone may miss important spatial detail. To address these complexities, the project evaluates distinct modeling strategies. First, Convolutional Neural Networks (CNNs) are used for image-based feature extraction from individual frames to identify handshapes. Second, to capture the sequential nature of ASL, we employ Long Short-Term Memory (LSTM) networks [10]. Unlike standard recurrent networks, LSTMs utilize specialized gating mechanisms to retain information over long time steps, making them robust baselines for modeling the temporal evolution of hand gestures. Third, we implement Transformers to model the temporal sequence and dependencies between frames using self-attention mechanisms. Finally, we explore hybrid architectures to integrate the feature identification strength of the CNN with the temporal modeling capabilities of both LSTMs and Transformers [6].

2 Related Work

Mishra et al. (IASC 2023) present a three-stage pipeline for hand gesture detection, tracking, and recognition, focusing on robustness across lighting, background, and user variation [8]. Detection uses a combination of skin color modeling and edge detection, while tracking is guided by inter-frame spatial proximity, often using a Kalman Filter. Recognition relies on template matching, comparing candidates against gesture prototypes at the pixel level. This paper is a helpful reference for foundational gesture recognition pipelines, especially when discussing segmentation and spatial-temporal modeling.

Alsharif et al. (2023) conducted a study evaluating deep learning architectures for ASL alphabet recognition, confirming that the ResNet-50 model achieved the highest recognition accuracy across all tested models, strongly supporting the choice of that specific ResNet backbone in this project's CNNs [2]. This work details components in their structures which again influenced ours, such as batch normalization to help stabilize gradients, average pooling, and flattening. Finally, the incorporation of the ReLU activation function and the dropout regularization technique is noted as crucial for improving feature recognition and reducing overfitting.

In Sign Language Recognition with Transformer Networks, De Coster et al. introduce four methods for sign language recognition [5], in which we focus on the two transformer-based architectures:

the Pose Transformer Network (PTN) and the Video Transformer Network (VTN). Each is tailored to different inputs. The PTN leverages 2D keypoints extracted via OpenPose. In contrast, the VTN processes features that were first extracted using a 2D CNN. The results show that PTN achieves competitive accuracy, but VTN demonstrates superior performance, especially on visually complex signs where keypoint extraction may fail to capture details. However, VTN requires more computational resources and data to generalize effectively. Due to this, the PTN architecture for gesture-based sign recognition provided more inspiration for this project’s structure, despite its slightly poorer performance. It allows our system to be modular and have a fallback system, which is extremely beneficial considering our limited resources. Future development for this project could explore the use of the VTN architecture for greater performance.

Brettmann et al. (2025) propose using Video Vision Transformers (ViViT), specifically VideoMAE, for word level ASL recognition [4]. They observe that while CNNs are computationally intensive and can struggle with global temporal dependencies, ViViTs utilize self-attention to effectively capture long range spatial and temporal relationships in video sequences. This work is relevant since it validates the choice of using a transformer backbone in ASL classification models and notes the significance of linear projection and positional embeddings for spatio-temporal representations. They also emphasize the importance of sampling evenly distributed frames to better capture temporal dynamics, informing the video frame input processing.

A two-stream ensemble transformer architecture for Isolated Sign Recognition (ISR) is proposed by Alkhoraf et al. (2025), combining posed-based and appearance-based methods [1]. This model architecture provides strong guidance for our hybrid model by demonstrating that combining appearance features with pose features achieves promising results. They convert 3D pose landmarks into RGB image representation for the pose stream, which can be more skilled at capturing subtle cues necessary for nuanced sign gesture recognition. This specialized pose processing informs the implementation of the CNN feature extractor that feeds into the pose transformer. Additionally, this study shows that using 16 frames from each video input balances performance and efficiency for video processing well, which was implemented in the data processing phase of this project.

A second ViViT model design for modeling dependencies across spatio-temporal tokens was introduced by Arnab et al. (2021) [3]. Their methodology highlights the importance of positional embeddings for maintaining sequence order and suggest initializing them by repeating image positional embeddings temporally. Crucially, they emphasize the use of initializing transformers with pretrained image models to effectively train larger models on smaller datasets.

3 Methods

3.1 Data Preprocessing

To prepare the datasets for American Sign Language classification, both static images of letters and video recordings of words were collected, organized, and transformed into formats suitable for PyTorch based modeling. The images data for classifying ASL letters came from the Kaggle ASL Alphabet [9]; they were downloaded directly as a nested folder where each subfolder was a letter containing 1,500 images of hands signing that letter, with a variety of lighting, angles, and skin tones to represent different people’s hand shapes and colors. A separate folder with training, testing, and validation subsets was created using an 80/10/10 split respectively. To fill these folders, each subfolder in the downloaded data was iterated through to collect file names for each letter class, which were then shuffled to randomize sample distribution across data splits. Each data split folder holds subfolders for each letter containing images with a standardized naming pattern. These images were all resized to 128x128 pixels and converted to PyTorch tensors.

The video frames for classifying ASL words come from video files from the Hugging Face American-Sign-Language-Dataset by ZahidYasinMittha [12]. The huggingface_hub API was queried to retrieve every .mp4 file in the dataset repository. Single character labels were filtered out to avoid overlap with the letters dataset and the top 40 most common words were selected based on the English Word Frequency Kaggle dataset [14] which contains metrics for how frequently a word occurs in the English language, as focusing on the most frequent words allows the model to prioritize commonly used vocabulary for practical applications. Sixteen evenly spaced frames were extracted from each video and saved in sequential order with standardized names. Then, in a similar fashion

to the image data, the video frames were split into training, testing, and validation sets, resized to 128x128 pixels, and converted to a PyTorch tensor.

3.2 MediaPipe

To obtain a compact and consistent representation of hand motion for each video frame, we used MediaPipe Hands to extract 2D keypoints for both the left and right hands. MediaPipe provides 21 landmarks per hand, each represented by (x, y) image coordinates, resulting in a 42-dimensional keypoint vector per frame when both hands are visible (and appropriately padded when only one hand is detected). Prior to model training, each video was processed frame-by-frame to detect hand landmarks, normalize coordinate ranges, and assemble the resulting vectors into fixed-length sequences. This preprocessing pipeline ensures that all downstream models receive temporally ordered, uniformly structured keypoint tensors reflecting framewise hand poses.

Because MediaPipe occasionally fails to detect one or both hands (e.g., due to motion blur or occlusion), the extraction pipeline includes robustness handling such as zero-padding missing detections and smoothing coordinates across time to reduce jitter. The resulting sequences serve as the input to both the LSTM and Transformer models used in this project. Keypoint extraction is lightweight compared to image-based CNN processing.

3.3 Convolutional Neural Network

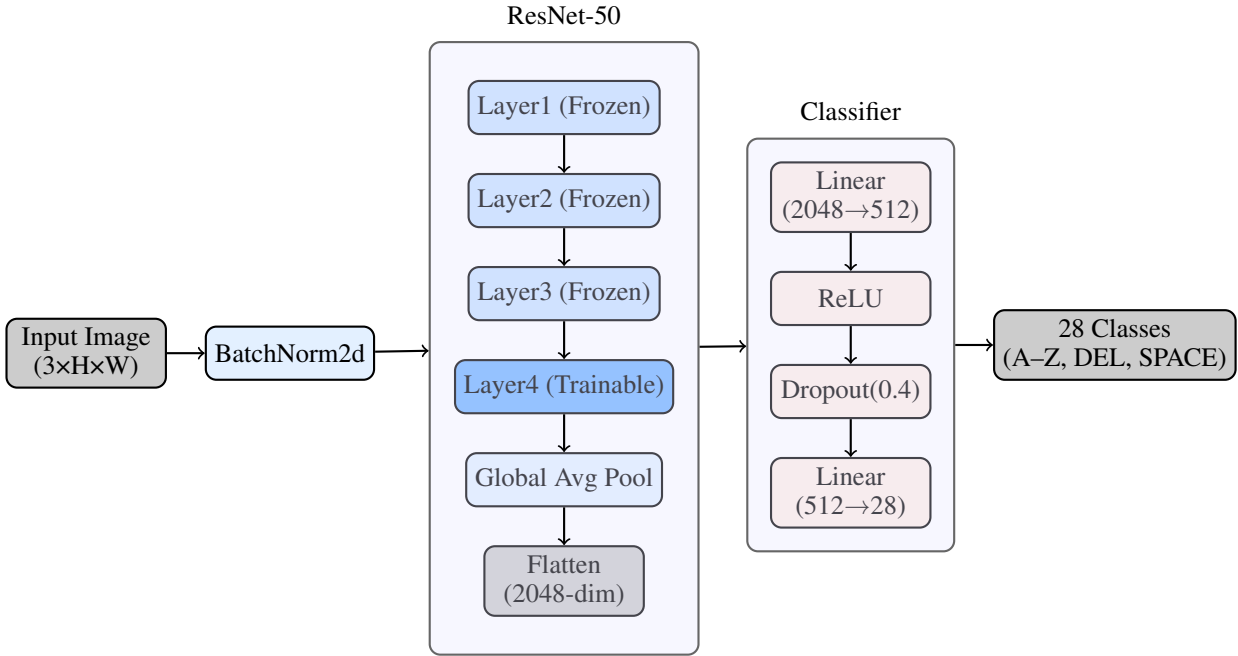


Figure 1: Detailed architecture of the ASL_CNN model.

The primary model for static American Sign Language (ASL) image recognition is a convolutional neural network (CNN) based on transfer learning from ResNet-50. The network first applies batch normalization to the input images to standardize the data distribution, followed by the ResNet-50 backbone, with all layers frozen except for the last residual block (layer4) to allow fine-tuning on ASL-specific features. The original fully connected layer is replaced with a custom classification head consisting of a linear layer projecting to 512 units, a ReLU activation, dropout with a rate of 0.4, and a final linear layer mapping to the 28 output classes (A–Z, DEL, SPACE). This design enables the network to leverage pretrained feature representations while adapting to the ASL static sign classification task. A feature-extractor variant of this model (ASL_CNN_FeatureExtractor) is also defined, which leaves out the final classification layer and outputs 512-dimensional feature vectors for use in our transformer model. The complete CNN architecture is illustrated in Figure 1.

Training was performed using stochastic gradient descent with momentum at two distinct learning rates: a smaller learning rate ($1e-4$) for the pretrained ResNet-50 block to avoid disrupting the pretrained weights and a standard learning rate ($1e-2$) for the randomly initialized fully connected layers. An exponential learning rate scheduler with a gamma of 0.9 was applied over 10 epochs. The training and validation datasets were loaded using PyTorch DataLoaders with a batch size of 32. Cross-entropy loss was used as the objective function. The model parameters resulting in the best validation accuracy were saved for later evaluation.

3.4 Long Short Term Memory

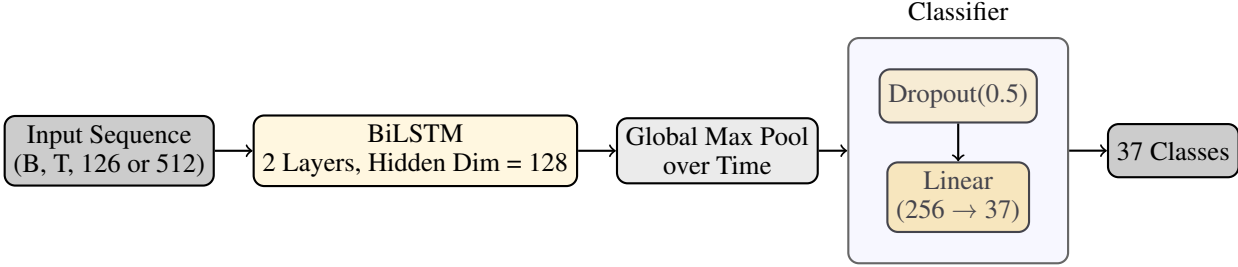


Figure 2: Detailed architecture of the HandLSTM model.

The first sequence model we evaluated was a bidirectional Long Short-Term Memory (LSTM) network designed to classify short sequences of MediaPipe keypoints or CNN extracted features corresponding to common English words. The model takes as input a sequence of 126- or 512-dimensional vectors, depending on which input it is getting, and processes them using a two-layer bidirectional LSTM with a hidden dimension of 128. After the recurrent layers, the model applies global max pooling across time. This pooled representation is passed through a dropout layer and a fully connected classifier that outputs probabilities over the 37 word classes. The full LSTM architecture is illustrated in Figure 2.

Training the LSTM model was performed using batches of padded keypoint sequences loaded through a PyTorch DataLoader. The model was optimized using Adam and a ReduceLROnPlateau learning rate scheduler. The training loop evaluated validation accuracy after each epoch, and the checkpoint with the highest validation performance was saved for testing.

3.5 Transformer

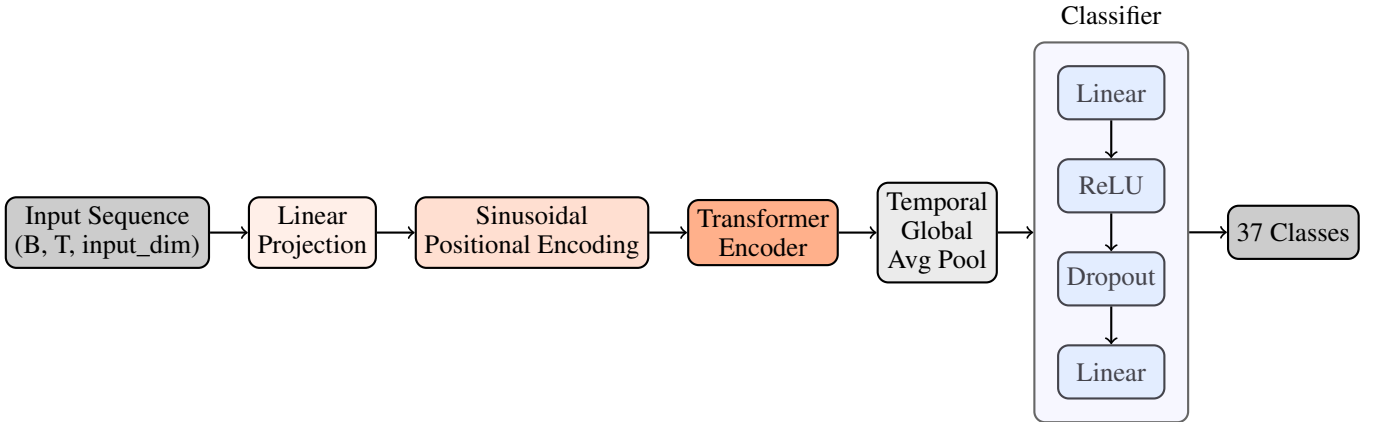


Figure 3: Detailed architecture of the KeypointTransformer model.

The other component of our system is a Transformer-based architecture designed to classify short sequences of keypoint vectors or learned CNN feature embeddings. The core model begins by

projecting each frame’s input representation (either 2D pose keypoints or 512-dimensional CNN features) into a latent space of dimension d_{model} via a linear projection layer. A sinusoidal positional encoding module is then added to preserve the temporal ordering of the sequence before it is processed by a stack of Transformer encoder layers. Each encoder layer consists of multihead self-attention and feedforward submodules, implemented using PyTorch’s `nn.TransformerEncoderLayer`. After the final encoder layer, global average pooling across the temporal dimension is applied to aggregate information over the entire sequence, producing a fixed-length representation that is fed into a custom two-layer classification head with ReLU activation and dropout regularization. The full Transformer architecture is illustrated in Figure 3. Training of the Transformer model followed a similar structured pipeline to the LSTM but leveraged the model’s ability to capture long-range dependencies using self-attention. Keypoint sequences were loaded using PyTorch DataLoaders, normalized, and padded to a fixed length. The model was trained using the Adam optimizer with dropout applied in both the encoder layers and the classifier to mitigate overfitting. The CosineAnnealingLR learning rate scheduler was used, providing a gradual learning rate decrease. Validation accuracy was evaluated each epoch, and the best-performing model was checkpointed for downstream comparison.

3.6 CNN–Transformer-LSTM Hybrid Model

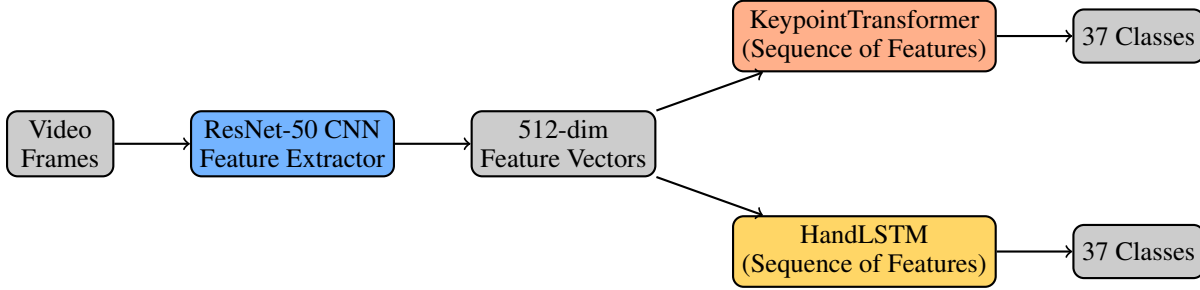


Figure 4: CNN–Transformer-LSTM architecture.

In addition to the standalone Transformer (Section 3.5, Figure 3), we define a hybrid CNN-Transformer-LSTM architecture that processes video sequences of ASL gestures by combining spatial feature extraction with temporal modeling. The CNN component is identical to the standalone CNN, with ResNet-50 pretrained weights, frozen early layers to retain pretrained features, and only the last residual block and fully connected layers are trainable. The fully connected classifier consists of a linear layer that reduces input features to 512 dimensions, followed by a ReLU activation, a dropout regularization layer, and a final linear layer projecting to the number of classes. The CNN is trained using the cross entropy loss function, optimized with AdamW weight decay, and a cosine annealing learning rate. During this hybrid model preparation, video frames are passed through this CNN to extract 512 dimensional feature embeddings.

The Transformer models the temporal dynamics of video sequences using the frame level features extracted by the CNN, while the LSTM also models this temporal evolution of video frame features but with a recurrent approach rather than self attention. The Transformer component is structured to take in sequences of the CNN-extracted frame features as input. Each feature vector is first linearly projected into a lower dimensional embedding, and positional encodings are added to account for temporal order of frames. The Transformer consists of a single layer encoder and applies a multi-head self attention encoder to capture long range temporal dependencies across the entire sequence. This is followed by global average pooling across the sequence. The output is passed through a classifier composed of layer normalization, dropout regularization, and a linear layer to produce class predictions. The model is trained using the same cross entropy loss function, AdamW optimizer with weight decay, and cosine annealing learning rate scheduler as the CNN.

The LSTM models the temporal structure of the data using a recurrent approach, processing the feature sequence step by step while maintaining a hidden state that carries the information through time. It uses the same CNN extracted features as input sequences. It consists of a single layer LSTM that process the temporal sequence of features. The last hidden state of the LSTM summarizes the

sequence and is passed through a dropout layer and linear classifier to generate predictions. To improve generalization, Gaussian noise is added to the input features during training which simulates feature variability. The LSTM is trained with cross entropy loss, AdamW optimizer, and cosine annealing learning rate scheduler, similar to the CNN and Transformer components. While the Transformer excels at capturing global temporal relationships across all frames, the LSTM encodes sequential and local temporal patterns while improving generalization abilities. Together, these sequence models use the CNN-extracted features to perform robust classification of ASL words.

3.7 CNN-LSTM Hybrid Model

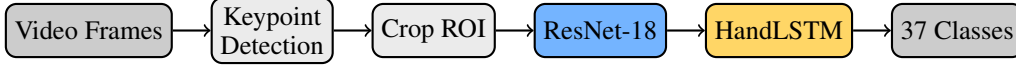


Figure 5: CNN feature extractor and LSTM hybrid architecture.

To model dynamic hand motion at the frame level, we trained an LSTM classifier on features extracted from cropped hand images. Each video is processed using MediaPipe Hands to detect the signing hand, and each frame is cropped tightly around the detected hand keypoints. This ensures that the system focuses only on the regions of interest. These cropped images were resized and normalized before being passed to a lightweight convolutional neural network for feature extraction. Demonstrations for cropping two example words are shown below.

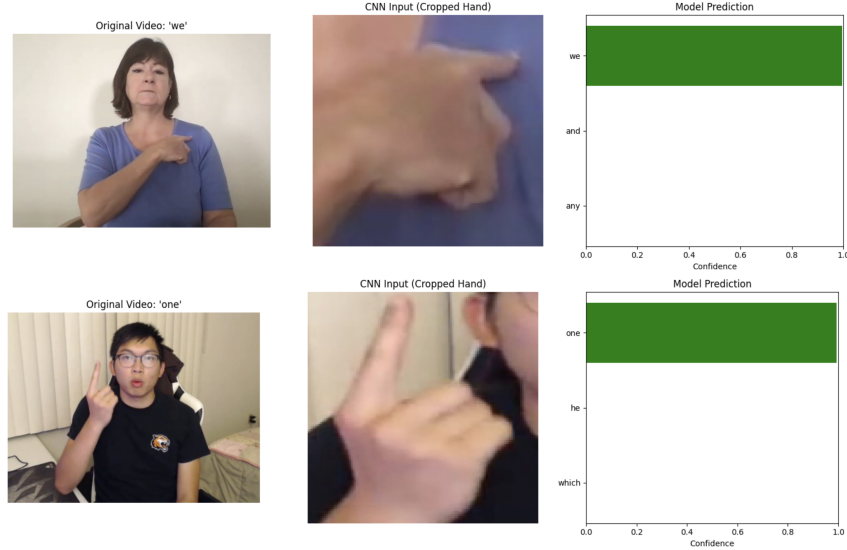


Figure 6: Cropped Hands Demonstration.

Unlike the static-sign classifier described in Section 3.3, which is based on a ResNet-50 backbone, this pipeline uses a lighter-weight ResNet-18 model initialized with ImageNet-pretrained weights. Only the final fully connected layer is replaced to produce a compact feature representation suitable for sequence modeling. The lightweight nature of this model allows frame-wise feature extraction to be performed efficiently, even on lower-end hardware, while still capturing sufficient spatial information about hand shape.

The output feature sequence from this CNN is fed into the HandLSTM (Section 3.4, Figure 2). During training, full sequences from each video clip were passed to the network, and global max pooling over the temporal dimension was applied to obtain a fixed-length representation. The model was trained using cross-entropy loss and the Adam optimizer with dropout applied both inside the LSTM and in the final classification head. This approach enabled the LSTM to learn temporal patterns of hand movement while relying on the lightweight CNN to extract compact, task-specific spatial features from each frame. Training is performed with cross-entropy loss using the Adam optimizer, with both

the CNN and LSTM updated end-to-end. Validation accuracy was evaluated each epoch, and the best-performing model weights were saved.

4 Experiments & Results

4.1 CNN for ASL Letter Classification

The standalone CNN model was trained on the ASL letter image dataset using a standard supervised learning pipeline. Each input image was resized to 128x128 pixels and normalized before being fed into the network. The training was performed for only 3 epochs using the cross entropy loss function, Adam optimizer with an initial learning rate of 1e-4, and a step-wise learning rate scheduler reducing the rate by every epoch. The dataset was divided into mini-batches of 16 samples, allowing the model to update its weights incrementally after each batch. For each batch, the loss and accuracy were computed, providing fine-grained feedback on model performance and helping to stabilize training. During training, both training and validation phases were monitored with loss and accuracy computed for each batch and aggregated across each epoch. The model achieving the highest validation accuracy was saved as the best model which was later loaded for evaluation. On the test set, the trained model achieved an overall accuracy of 99.62% on the test dataset and a validation accuracy of 99.57%. The confusion matrix below shows the how skilled the model is at correctly classifying American Sign Language letters based on input images of hand symbols.

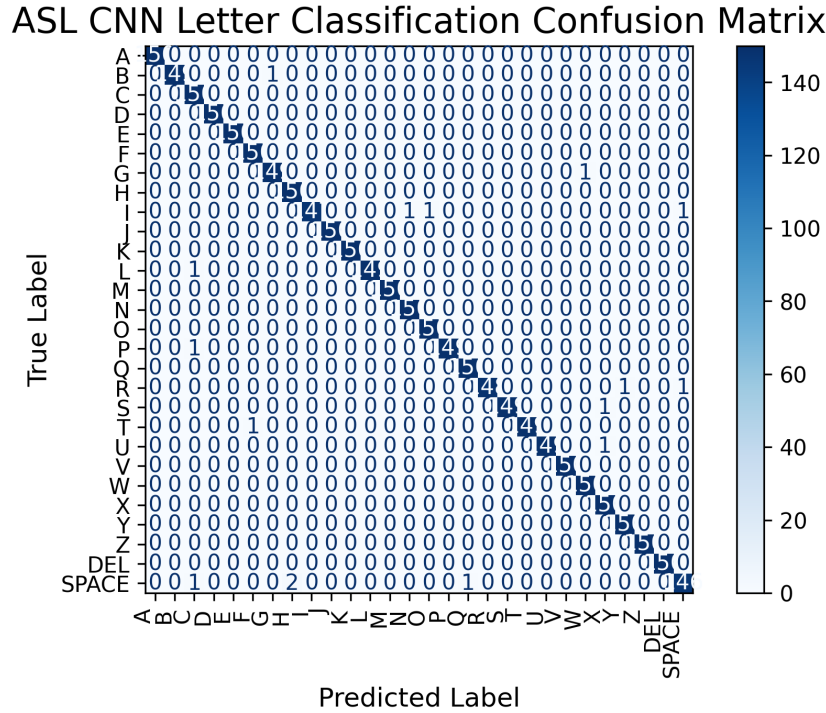


Figure 7: ASL CNN Letter Classification Confusion Matrix.

4.2 LSTM and Transformer Models for Word Classification

We evaluated the two sequence models trained on MediaPipe-extracted keypoints: the HandLSTM and the KeypointTransformer. Both models were trained on the same dataset of pose keypoint sequences, using a batch size of 32 for 100 epochs. During training, we monitored training and validation accuracy at each epoch.

The LSTM achieved a training accuracy of 94.4% and a validation accuracy of 77.1%, indicating overfitting, as the model learned the training distribution well but did not maintain the same performance

on unseen data. The Transformer model performed worse, reaching 51.2% training accuracy and 49.5% validation accuracy, suggesting that the Transformer was unable to effectively learn temporal relationships from sparse keypoint vectors, especially with the relatively small dataset and short sequence lengths, whereas the LSTM was better suited to this simpler input representation. Final evaluation on the test set produced test accuracies of 69.2% and 54.7%, respectively.

4.3 CNN-Transformer-LSTM Hybrid Model for ASL Word Classification

The CNN was trained to classify individual frames of videos representing ASL words on 20 epochs with batch sizes of 64. The training accuracy increased steadily, ultimately reaching 97.3% accurate classifications. However, the highest validation accuracy was 9% which suggests overfitting, likely due to limited frame level information for word classification. While the CNN learned frame level patterns well, the frame-by-frame approach alone was insufficient for accurate word classification.

The Transformer model was trained to classify sequences of CNN extracted frame features into words over 50 epochs and a batch size of 32. Similarly, the final training accuracy was very high (99.9%), but the validation and test accuracies were low (6% and about 2.6% respectively). These results indicate poor generalization, despite high training accuracy. CNN features alone were insufficient to allow the transformer to generalize to unseen videos.

The LSTM model was trained over 50 epochs with a batch size of 32 on sequences of CNN extracted features with added noise to prevent overfitting and improve generalization. The training accuracy grew steadily, ultimately reaching 97.7%. Validation accuracy was highest at 7.62% and the overall testing accuracy was 3.42%. The validation and testing results proved to be more promising than that of the Transformer, but still not robust enough for real life application.

4.4 CNN-LSTM Hybrid Model for Word Classification

The MediaPipe assisted CNN-LSTM framework showed promising results for classifying ASL words. Cropped hand images were generated from training and validation videos using MediaPipe's hand landmark detector. For each video, ten evenly spaced frames were taken and valid hand crops were resized to 128x128 pixels. A pretrained ResNet18 was then used to fine tune the cropped images. The CNN was trained for ten epochs with a batch size of 64, using data augmentation to improve robustness to lighting and pose variations. Each cropped image was passed through the trained CNN to obtain a sequence of 512 dimensional feature vectors representing the hand's visual appearance over time. These feature sequences were then used to train a two-layer LSTM with global max pooling across time. The LSTM was trained for 30 epochs to classify ASL words using only temporal structure in the extracted feature sequences. Training the CNN on hand images showed steady improvements in validation accuracy across the epochs, starting at 23.2% and ending with an accuracy of 50.9%. This indicates that despite the limited number of training samples per class, the CNN was able to learn discriminative spatial representations of hand shapes that generalize moderately well to unseen cropped images. Using the CNN as a feature extractor, the LSTM model achieve substantially higher performance by leveraging temporal dynamics across video frames. Validation accuracy increased quickly during early epochs, reaching 81.9% by the 8th epoch, and had a final testing accuracy of 75.21%. This demonstrates that the combined CNN LSTM with MediaPipe architecture was able to successfully learn representations of both static hand appearance and temporal motion patterns.

5 Conclusion

5.1 Interpretation of Results

The CNN model demonstrated particularly strong performance on static signs (letters A-Z, DEL, and SPACE), achieving 98.31% accuracy on the test data, indicating that the model effectively captured spatial features such as hand shape, finger positioning, and orientation. The ResNet-50 transfer learning approach proved beneficial, as the pretrained weights on ImageNet provided robust feature representations that translated well to sign recognition.

Our hybrid CNN-Transformer-LSTM architecture achieved 3.42% on the ASL classification task, demonstrating disappointing performance in distinguishing between 37 sign classes. The word CNN

was effective in learning frame level visual features as shown by the high training accuracy. The low validation accuracy, however, indicates that these individual frames lack the temporal context necessary to differentiate between words, leading to overfitting on the training. The Transformer aspect of the hybrid model, which was designed to use temporal sequences of CNN extracted features, also showed high training accuracy but poor generalization abilities. Despite the model’s capacity to capture sequential dependencies, the CNN features did not provide enough discriminative information across frames for the transformer to accurately classify unseen sequences. This highlights the limitations of using frame level features without additional temporal or structural information. The LSTM part of the hybrid model, which also operated on sequences of CNN features, also incorporated feature level noise to prevent overfitting. Since the training accuracy was very high and the validation and test accuracies were slightly higher than the Transformer, it suggests that sequential modeling with noise offered modest improvement in generalization. Nevertheless, the results remained insufficient for practical deployment, indicating that further enhancements, such as richer feature representations or larger datasets, are necessary to achieve reliable ASL word recognition.

The results of the CNN-LSTM with MediaPipe model show that relying on individual cropped hand images alone is insufficient for high accuracy world level ASL classification, which is consistent with the results from our other models. However, once the CNN’s spatial features were integrated into a sequence and processed through the LSTM, overall classification performance improved substantially. The LSTM was able to capture the temporal evolution of hand configurations, leading to a high testing accuracy which was an especially strong result given the limited dataset size, relatively simple architecture, and other model results. This implies that the additions made by MediaPipe, specifically its reliable hand localization, were very influential in contributing to the high classification accuracy because they ensured that the CNN consistently received well centered, hand focused inputs, reducing background noise and pose variability. These results indicate that pretrained CNN features, LSTM sequence modeling, and MediaPipe based hand detection and cropping pipeline together form an effective approach for video based ASL word recognition.

5.2 Limitations

Several limitations must be acknowledged when interpreting these results. First, the dataset used for training and evaluation has several constrained conditions with inconsistencies between data types. Static letter sign images were pre-cropped to the hand region with minimal background clutter, while dynamic gesture sign videos captured the upper body and head area. This is a significant domain shift problem that hurt performance as seen in the low accuracy of the CNN-Transformer-LSTM architecture and higher accuracy of the CNN-LSTM architecture. Although cropping frames to the region of interest improved accuracy, spatial information crucial to the definition of certain gesture signs is lost.

All videos were shot from a consistent head-on angle with fairly uniform lighting, and each gesture sign was represented by only approximately twenty-four short video clips of a few seconds each. Performance may degrade when the system encounters different lighting conditions, complex backgrounds, varying camera angles, or differently framed inputs, as these factors were not extensively represented in our training data and could potentially limit the model’s ability to generalize.

Additionally, our evaluation was limited to isolated signs rather than continuous signing, which represents a significant simplification of natural ASL communication. Real conversational ASL involves coarticulation effects, where signs blend together, as well as non-manual markers such as facial expressions and body posture, which our current system does not capture or analyze [15, 16].

5.3 Advantages and Disadvantages

The transfer learning approach using ResNet-50 provided several key benefits. It significantly reduced training time compared to training from scratch, required less data to achieve convergence, and leveraged rich feature representations learned from over a million images. This made the project computationally feasible within our resource constraints. With a large enough dataset, the Transformer architecture’s self-attention mechanism can be effective for identifying relevant spatial regions and temporal segments without explicit spatial annotations.

The hybrid CNN-LSTM architecture offered complementary advantages for temporal sequence modeling. LSTMs are well-established for handling sequential data and have been successfully

applied to gesture recognition tasks due to their ability to maintain long-term dependencies [11]. The LSTM's recurrent structure processes frames sequentially, naturally capturing the temporal ordering of sign language gestures. This sequential processing aligns intuitively with how signs unfold over time, making the model architecture conceptually straightforward.

However, the LSTM approach revealed several practical limitations in our experiments. Training the model on cropped frames required a reliable hand detection and tracking system as a preprocessing step, introducing an additional point of failure and computational overhead. On the other hand, the full-frame model had to learn to focus on relevant hand regions and ignore background clutter, which it struggled with. To achieve comparable performance would require more training data, which is not always readily available.

The architecture's use of transfer learning from ResNet-50 provides some flexibility for future modifications. Although the CNN feature extractor was trained jointly with the Transformer and LSTM as a unified model, the pretrained backbone could potentially be replaced with alternative architectures such as EfficientNet, which has been shown to have better accuracy with fewer FLOPS [13]. However, such substitutions would require retraining the entire pipeline, as the LSTM and Transformer learn to process features specifically extracted by a CNN. This coupling between components means that improvements to one part of the architecture may require adjustments to the other.

Additionally, the Transformer component has an inherent efficiency problem: its self-attention mechanism has a "quadratic running time in terms of the number of input tokens" [7]. Analyzing longer gesture sequences becomes increasingly slow and resource-intensive, limiting our ability to recognize complex signs that require more frames to capture the full motion. Methods have been suggested to reduce complexity, but Duman Keles et al. [7] stated that they all have some undesirable errors.

5.4 Future Improvements and Research Directions

Expanding the model to recognize non-manual markers would significantly improve its linguistic accuracy. Incorporating facial expression analysis and body posture tracking could enable the system to capture the grammatical information conveyed through these channels in ASL.

Data augmentation strategies specifically designed for sign language could improve generalization. Collecting more data from signers across different demographics, signing styles, and regional variants could address representation biases.

Finally, a critical next step would be moving toward continuous sign recognition rather than isolated sign classification. Developing this capability would require the incorporation of some kind of video segmentation method for identifying temporal boundaries. Achieving this would represent a major step toward creating practical tools that could assist in real-time communication for the deaf and hard-of-hearing community.

References

- [1] Abdulelah Ali Alkhoraif, Mansour Alsulaiman, Wadood Abdul, and Mohamed Bencherif. Ensemble transformer-based word-level sign language recognition with multi-modal input fusion. *Journal of Engineering Research*, 2025.
- [2] Ali Salem Altaher Ahmed Altaher Mohammad Ilyas Alsharif, Bader and Easa Alalwany. Deep learning technology to recognize american sign language alphabet. *Sensors*, 23(18):7970, 2023.
- [3] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lucic, and Cordelia Schmid. Vivit: A video vision transformer. 2021.
- [4] Alexander Brettmann, Jakob Grävinghoff, Marlene Rüschoff, and Marie Westhues. Breaking the barriers: Video vision transformers for word-level sign language recognition. 2025.
- [5] Mathieu De Coster, Mieke Van Herreweghe, and Joni Dambre. Sign language recognition with transformer networks. In *Proceedings of the 12th International Conference on Language Resources and Evaluation (LREC 2020)*, pages 6018–6024, 2020.
- [6] Jeffrey Donahue, Lisa Anne Hendricks, Sergio Guadarrama, Marcus Rohrbach, Subhashini Venugopalan, Kate Saenko, and Trevor Darrell. Long-term recurrent convolutional networks for

- visual recognition and description. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2625–2634, 2015.
- [7] Feyza Duman Keles, Pruthvi Mahesakya Wijewardena, and Chinmay Hegde. On the computational complexity of self-attention. In Shipra Agrawal and Francesco Orabona, editors, *Proceedings of The 34th International Conference on Algorithmic Learning Theory*, volume 201 of *Proceedings of Machine Learning Research*, pages 597–619. PMLR, 2023.
 - [8] S Gnanapriya and K Rahimunnisa. A hybrid deep learning model for real time hand gestures recognition. *Intelligent Automation & Soft Computing*, 36(1), 2023.
 - [9] Grassknoted. Asl alphabet dataset. <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>, 2020.
 - [10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
 - [11] Jiayu Huang and Varin Chouvatut. Video-based sign language recognition via ResNet and LSTM network. *Journal of Imaging*, 10(6):149, June 2024.
 - [12] Zahid Yasin Mittha. American sign language dataset. <https://huggingface.co/datasets/ZahidYasinMittha/American-Sign-Language-Dataset>, 2021.
 - [13] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR, 2019.
 - [14] Ryan Tatman. English word frequency dataset. <https://www.kaggle.com/datasets/rtatman/english-word-frequency>, 2016.
 - [15] William Vicars. Coarticulation. <https://www.lifeprint.com/asl101/topics/coarticulation.htm>, n.d. ASL University.
 - [16] William Vicars. Non-manual markers in ASL (NMM’s). <https://www.lifeprint.com/asl101/pages-layout/nonmanualmarkers.htm>, n.d. ASL University.